```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import io

uploaded = files.upload()

filename = next(iter(uploaded))
df = pd.read_csv(io.BytesIO(uploaded[filename]))

print("Dataset loaded successfully!")
display(df.head())
```

Choose Files sales_data…practice.csv

sales_data_cleaning_practice.csv(text/csv) - 1348 bytes, last modified: 10/27/2026 - 100% done

```
Saving sales_data_cleaning_practice.csv to sales_data_cleaning_practice.csv
Dataset loaded successfully!
```

	order_id	product	quantity	price	region	order_date
0	1001	laptop	2.0	50000	south	2026-01-03
1	1002	Mouse	5.0	500	NORTH	2026-01-04
2	1003	keyboard	3.0	1500	East	2026-01-05
3	1004	Monitor	2.0	12000	west	2026-01-06
4	1002	Mouse	5.0	500	NORTH	2026-01-04

```
# First five rows
display(df.head())

# Dataset information
df.info()

# Number of rows and columns
print("Dataset shape:", df.shape)

# Statistical summary
display(df.describe(include="all"))
```

	order_id	product	quantity	price	region	order_date
0	1001	laptop	2.0	50000	south	2026-01-03
1	1002	Mouse	5.0	500	NORTH	2026-01-04
2	1003	keyboard	3.0	1500	East	2026-01-05
3	1004	Monitor	2.0	12000	west	2026-01-06
4	1002	Mouse	5.0	500	NORTH	2026-01-04

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 33 entries, 0 to 32
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   order_id    33 non-null     int64
 1   product     33 non-null     object
 2   quantity    32 non-null     float64
 3   price       32 non-null     object
 4   region      32 non-null     object
 5   order_date  33 non-null     object
dtypes: float64(1), int64(1), object(4)
memory usage: 1.7+ KB
Dataset shape: (33, 6)
```

	order_id	product	quantity	price	region	order_date
count	33.000000	33	32.000000	32	32	33
unique	NaN	24	NaN	18	11	32
top	NaN	Mouse	NaN	500	East	2026-01-04
freq	NaN	3	NaN	4	8	2
mean	1016.060606	NaN	4.343750	NaN	NaN	NaN
std	9.571897	NaN	8.597109	NaN	NaN	NaN
min	1001.000000	NaN	1.000000	NaN	NaN	NaN
25%	1008.000000	NaN	1.750000	NaN	NaN	NaN
50%	1016.000000	NaN	2.000000	NaN	NaN	NaN
75%	1024.000000	NaN	4.000000	NaN	NaN	NaN
max	1032.000000	NaN	50.000000	NaN	NaN	NaN

```
# Identify missing values
print("Missing values before cleaning:")
print(df.isnull().sum())

# Fill missing text values
```

```
df["region"] = df["region"].fillna("Unknown")

# Convert numeric columns before filling missing values
df["price"] = pd.to_numeric(df["price"], errors="coerce"
df["quantity"] = pd.to_numeric(df["quantity"], errors="c

# Fill missing numeric values with their column medians
df["price"] = df["price"].fillna(df["price"].median())
df["quantity"] = df["quantity"].fillna(df["quantity"].me

print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

```
Missing values before cleaning:
order_id      0
product       0
quantity      1
price         1
region        1
order_date    0
dtype: int64

Missing values after cleaning:
order_id      0
product       0
quantity      0
price         0
region        0
order_date    0
dtype: int64
```

```
# Clean product names
df["product"] = df["product"].astype("string").str.strip().str.title()

# Standardize regions
df["region"] = df["region"].astype("string").str.strip().str.upper()

# Clean order IDs if stored as text
df["order_id"] = df["order_id"].astype("string").str.strip()

display(df.head())
```

	order_id	product	quantity	price	region	order_date
0	1001	Laptop	2.0	50000.0	SOUTH	2026-01-03
1	1002	Mouse	5.0	500.0	NORTH	2026-01-04
2	1003	Keyboard	3.0	1500.0	EAST	2026-01-05
3	1004	Monitor	2.0	12000.0	WEST	2026-01-06
4	1002	Mouse	5.0	500.0	NORTH	2026-01-04

```
# Ensure numeric columns have numeric types
df["price"] = pd.to_numeric(df["price"], errors="coerce")
df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce")

# Create sample dates for practice
df["order_date"] = pd.date_range(
    start="2026-01-01",
    periods=len(df),
    freq="D"
)

# Convert the date column to datetime
df["order_date"] = pd.to_datetime(
    df["order_date"], errors="coerce"
)

# Verify data types
print(df.dtypes)
```

```
order_id      string[python]
product       string[python]
quantity             float64
price                float64
region        string[python]
order_date    datetime64[ns]
dtype: object
```

```
# Calculate IQR for price
Q1 = df["price"].quantile(0.25)
Q3 = df["price"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
# Identify outliers
outliers = df[
    (df["price"] < lower_bound) |
    (df["price"] > upper_bound)
]

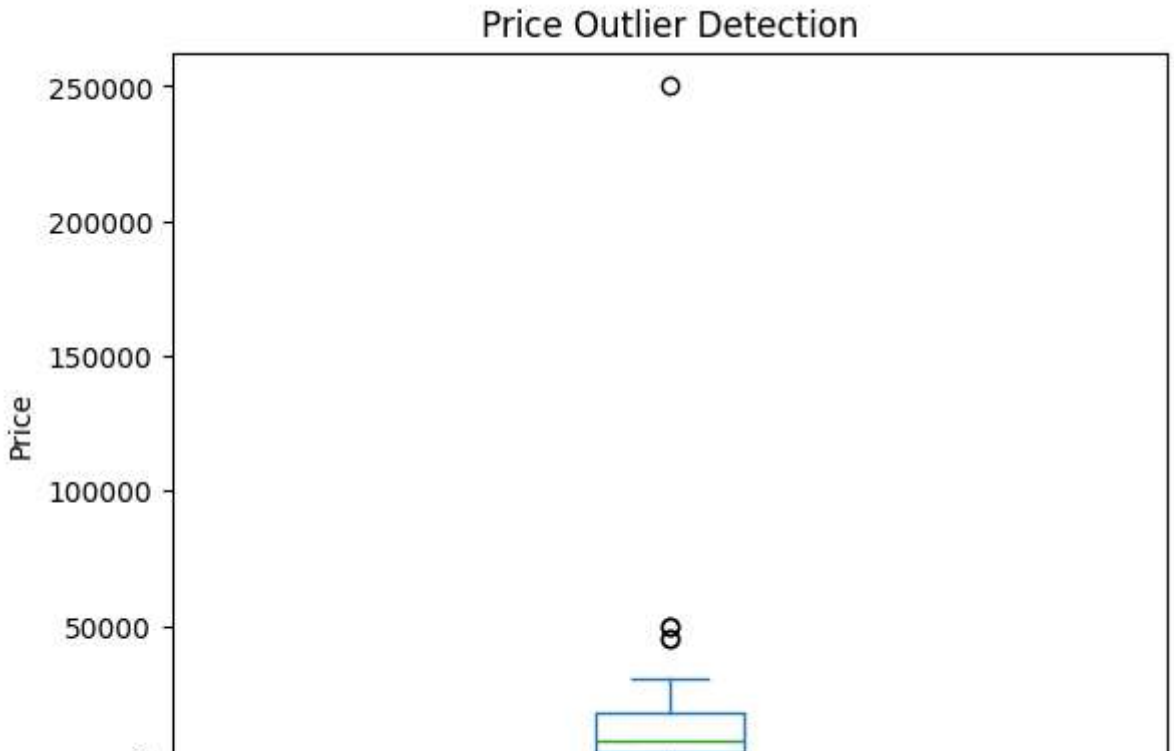
print("Lower bound:", lower_bound)
print("Upper bound:", upper_bound)
print("Number of price outliers:", len(outliers))

display(outliers)

# Visualize outliers
df["price"].plot(kind="box", title="Price Outlier Detection")
plt.ylabel("Price")
plt.show()
```

```
Lower bound: -20750.0
Upper bound: 41250.0
Number of price outliers: 5
```

	order_id	product	quantity	price	region	order_date
0	1001	Laptop	2.0	50000.0	SOUTH	2026-01-01
9	1009	Camera	2.0	45000.0	SOUTH	2026-01-10
24	1024	Laptop	1.0	50000.0	EAST	2026-01-25
28	1028	Camera	1.0	45000.0	EAST	2026-01-29
30	1030	Gaming Pc	1.0	250000.0	SOUTH	2026-01-31



Next steps: Generate code with outliers | New interactive sheet

```
df["revenue"] = df["price"] * df["quantity"]

print("Revenue column created!")
display(df[["order_id", "product", "price", "quantity", "revenue"]].head())
```

```
Revenue column created!
```

	order_id	product	price	quantity	revenue
0	1001	Laptop	50000.0	2.0	100000.0
1	1002	Mouse	500.0	5.0	2500.0
2	1003	Keyboard	1500.0	3.0	4500.0
3	1004	Monitor	12000.0	2.0	24000.0
4	1002	Mouse	500.0	5.0	2500.0

```
def classify_order(revenue):
    if revenue >= 50000:
        return "High Value"
    elif revenue >= 10000:
        return "Medium Value"
    else:
        return "Low Value"

df["order_category"] = df["revenue"].apply(classify_order)

display(
    df[["order_id", "revenue", "order_category"]].head(10)
)
```

	order_id	revenue	order_category
0	1001	100000.0	High Value
1	1002	2500.0	Low Value
2	1003	4500.0	Low Value

```
# Export cleaned and transformed data
df.to_csv("processed_sales.csv", index=False)

print("Processed dataset saved successfully!")
```

```
Processed dataset saved successfully!
```

7	1007	7500.0	Low Value
8	1008	8000.0	Low Value

```
files.download("processed_sales.csv")
```